

Return a Value from a Function

ELEC1006: ENGINEERING COMPUTING

The return Statement

- Used to end execution of a function.
- Can be placed anywhere in a function.
- Can be used to prevent abnormal termination of program (see Example 2).
- In a `void` function, where there is no `return` statement, the function ends at its last `}`.

1) Returning a Value from a Function

- In a value-returning function, the `return` statement can be used to return a value from function to the point of call. Example:

Return Type

```
int sum(int num1, int num2)
{
    double result;
    result = num1 + num2;
    return result;
}
```

Value Being Returned

2) Returning a Value from a Function

```
int sum(int num1, int num2)
{
    return num1 + num2;
}
```

Functions can return the values of expressions, such as `num1 + num2`



Example 1

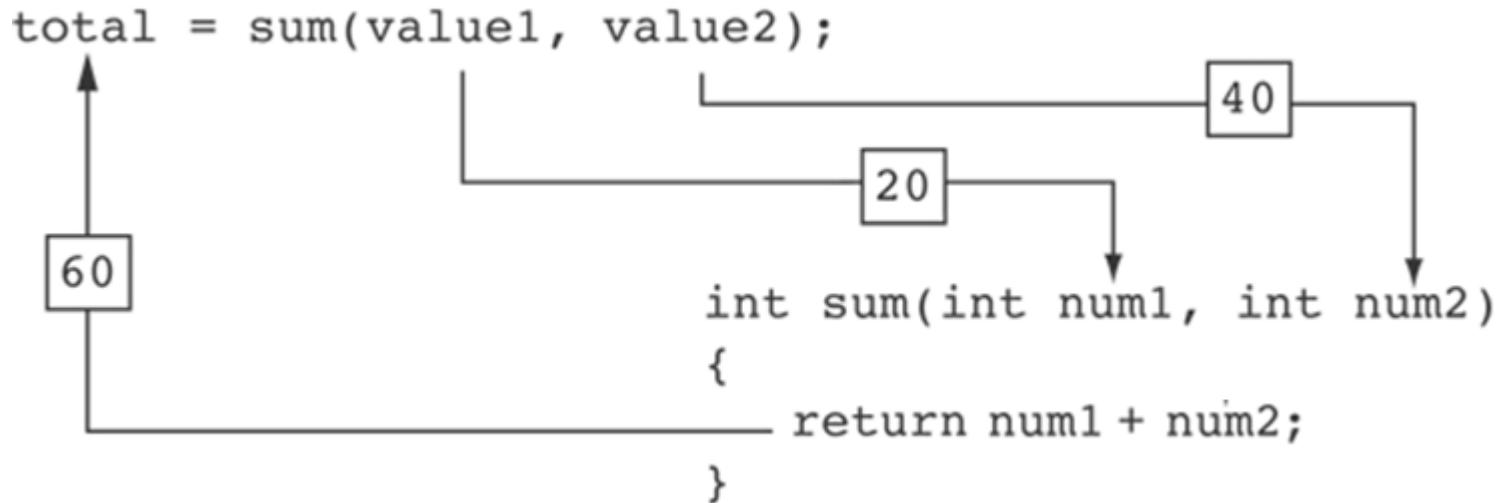
```
1 // This program uses a function that returns a value.
2 #include <iostream>
3 using namespace std;
4
5 // Function prototype
6 int sum(int, int);
7
8 int main()
9 {
10     int value1 = 20,    // The first value
11         value2 = 40,    // The second value
12         total;          // To hold the total
13
14     // Call the sum function, passing the contents of
15     // value1 and value2 as arguments. Assign the return
16     // value to the total variable.
17     total = sum(value1, value2);
18
19     // Display the sum of the values.
20     cout << "The sum of " << value1 << " and "
21         << value2 << " is " << total << endl;
22     return 0;
23 }
```

```
24
25 //*****
26 // Definition of function sum. This function returns *
27 // the sum of its two parameters.                    *
28 //*****
29
30 int sum(int num1, int num2)
31 {
32     return num1 + num2;
33 }
```

Program Output

The sum of 20 and 40 is 60

Example 1 continued ...



The statement in line 17 calls the `sum` function, passing `value1` and `value2` as arguments. The return value is assigned to the `total` variable.

3) Returning a Value from a Function

- The prototype and the definition must indicate the data type of return value (not `void`).
- Calling function should use return value:
 - assign it to a variable.
 - send it to `cout`.
 - use it in an expression.
- Statements that follow the `return` statement will not be executed. See Example 2.

Example 2

```

1 // This program uses a function to perform division. If division
2 // by zero is detected, the function returns.
3 #include <iostream>
4 using namespace std;
5
6 // Function prototype.
7 void divide(double, double);
8
9 int main()
10 {
11     double num1, num2;
12
13     cout << "Enter two numbers and I will divide the first\n";
14     cout << "number by the second number: ";
15     cin >> num1 >> num2;
16     divide(num1, num2);
17     return 0;
18 }
20 //*****
21 // Definition of function divide.
22 // Uses two parameters: arg1 and arg2. The function divides arg1
23 // by arg2 and shows the result. If arg2 is zero, however, the
24 // function returns.
25 //*****
26
27 void divide(double arg1, double arg2)
28 {
29     if (arg2 == 0.0)
30     {
31         cout << "Sorry, I cannot divide by zero.\n";
32         return;
33     }
34     cout << "The quotient is " << (arg1 / arg2) << endl;
35 }

```

Program Output with Example Input Shown in Bold

```

Enter two numbers and I will divide the first
number by the second number: 12 0 [Enter]
Sorry, I cannot divide by zero.

```


Returning a Boolean Value

- Function can return `true` or `false`
- Declare return type in function prototype and heading as `bool`
- Function body must contain `return` statement(s) that return `true` or `false`
- Calling function can use return value in a relational expression

Example 3

```

1 // This program uses a function that returns true or false.
2 #include <iostream>
3 using namespace std;
4
5 // Function prototype
6 bool isEven(int);
7
8 int main()
9 {
10     int val;
11
12     // Get a number from the user.
13     cout << "Enter an integer and I will tell you ";
14     cout << "if it is even or odd: ";
15     cin >> val;
16
17     // Indicate whether it is even or odd.
18     if (isEven(val))
19         cout << val << " is even.\n";
20     else
21         cout << val << " is odd.\n";
22     return 0;
23 }
24
25 //*****
26 // Definition of function isEven. This function accepts an
27 // integer argument and tests it to be even or odd. The function
28 // returns true if the argument is even or false if the argument
29 // is odd. The return value is a bool.
30 //*****
31
32 bool isEven(int number)
33 {
34     bool status;
35
36     if (number % 2 == 0)
37         status = true; // The number is even if there is no remainder.
38     else
39         status = false; // Otherwise, the number is odd.
40     return status;
41 }

```

Program Output with Example Input Shown in Bold

Enter an integer and I will tell you if it is even or odd: **5** [Enter]
5 is odd.

More info

- [1] cplusplus.com: Functions
<https://cplusplus.com/doc/tutorial/functions/>
- [2] learncpp.com: 2.2 – Function return values
<https://www.learncpp.com/cpp-tutorial/function-return-values/>